TEAM DECISION GUIDE | 14 OCTOBER 2026

Seven ideas. One project to build.

Nebius x NVIDIA Global AI Hackathon

A practical comparison of RuleForge, Blackout, Chimera, IncentiveLab, Morph, Treaty, and EventWeave.

Prepared so every teammate can understand the concepts, compare the engineering work, and choose a project without reading the chat history.

The current recommendation

RuleForge is ranked first; Blackout is the closest alternative. Chimera is the strongest additional candidate if the team wants a physical demonstration using basic devices. This ranking weighs the published rubric, the time remaining, and a software-heavy team with limited hardware. It is a judgment, not a prediction of a prize.

The opportunity

The event lists a **$20,000 grand prize** and a submission deadline of **30 October 2026, 10:00 AM PDT / 10:30 PM IST**. The required demonstration video is no longer than three minutes. A working entry must use at least one NVIDIA open-source model and run on Nebius Token Factory or Nebius AI Cloud. See the official rules before submission [S01].

How to use this guide

Read the comparison on page 2. Each idea then receives two pages: the first explains the product and business; the second explains the build, demonstration, evidence, and risks. Use the decision worksheet on page 18 to choose after a short technical experiment.

What is established and what is proposed

Event requirements and competitor capabilities are sourced. Product designs, buyer hypotheses, commercial models, build plans, and rankings are proposals. None has been validated by a customer pilot or working prototype in this conversation. No competitor scan here establishes that an idea is unique.

Scope: the seven ideas in the final ranked shortlist. Earlier rejected concepts are intentionally not included.

QUICK COMPARISON

The ranked shortlist

Ranked for this hackathon with the current team assumptions and deadline. The ranking is not an estimate of winning probability.

Rank / idea	What the audience sees	Main risk	Pages
1 RuleForge	A new game rule becomes playable; agents find an exploit and replay the fix.	Scripted discovery; weak game-designer value.	4-5
2 Blackout	Internet fails; the team keeps working and reconciles conflicting updates.	Network consistency; an incidental role for AI.	6-7
3 Chimera	Phones become an inspection station and recover from losing a viewpoint.	Calibration; false visual confirmations.	8-9
4 IncentiveLab	Agents exploit a promotion in a sandbox and compare repairs.	Incorrect policy model; unrealistic claims.	10-11
5 Morph	A changed robot searches for a controller that restores useful behavior.	Slow or unreliable adaptation.	12-13
6 Treaty	Separate agents negotiate an approved plan, then handle a withdrawn resource.	More complexity than a simple solver.	14-15
7 EventWeave	Conflicting recordings become a source-linked timeline with uncertainty.	Incorrect evidence interpretation.	16-17

Choose based on the kind of work your team wants

Interactive software: RuleForge or IncentiveLab.
Distributed systems: Blackout.
Cameras and physical interaction: Chimera.
Robotics research: Morph.
Constraints and negotiation: Treaty.
Multimodal evidence: EventWeave.

What could change this order

A credible pilot partner and early technical result can outweigh the current ranking. Blackout could become the lead with a real field-operations team. Morph could move up if a tested simulator and recovery method already exist in your team. A polished interface cannot rescue a core mechanism that does not work.

EVALUATION LENS

What an entry must prove

The official rubric names technological implementation, design, potential impact, and quality of the idea [S01]. Use these as evidence categories, not as a reason to add more features.

Criterion	Evidence to bring
Technological implementation	An end-to-end working system; a meaningful NVIDIA model and Nebius role; logs and replayable results; measured latency or compute where relevant.
Design	A person can understand the task, act, inspect results, and recover from a failure. The entry feels like one coherent product.
Potential impact	A specific audience, a real workflow, and evidence that the product improves it. User feedback and measured comparisons beat hypothetical market size.
Quality of the idea	A clear contribution beyond existing tools, demonstrated on a challenge that was not hardcoded into the demo.

How the judge research informs this guide

Public work from the panel supports emphasizing practical evaluation, production-quality execution, feedback from real traces, and constrained agent capabilities [S10-S13]. This is an inference about persuasive evidence, not knowledge of any judge's preferences or scoring. The same solid evidence package should serve the whole panel.

A simple architecture rule

Let the model interpret ambiguous information and propose useful actions. Let ordinary software enforce permissions, track exact state, and check outcomes. This gives every idea a visible boundary between a suggestion and a verified result.

Small glossary

Agent: software that uses a model to choose actions and tools.
Sandbox: an isolated test environment, not a live customer system.
Baseline: a simpler or existing method used for comparison.
Held-out case: a test not used while tuning the system.
Regression test: a saved test that checks whether an old failure returns.
Deterministic check: ordinary code that gives the same result for the same defined input.
Network partition: groups of devices temporarily cannot communicate.

Submission basics

Prepare the working demo, public licensed repository, reproducible setup, project description, platform feedback, and a public video of at most three minutes. Physical AI has a specific one-minute operating-hardware or application-module footage requirement. Check the complete current rules for eligibility and all details [S01].

CANDIDATE 1 | PRODUCT AND OPPORTUNITY

RuleForge

A laboratory for inventing and stress-testing game mechanics

> **Invent a rule. Watch agents find a way to break it. Repair it, then play.**

In plain language

A game designer describes a new rule in ordinary language. RuleForge turns it into a playable rule inside a supported game engine. Automated players try different strategies, reveal exploits, and save a replay. The designer decides whether to change the rule and can test the revision immediately.

A concrete example

Imagine a turn-based arena where players may borrow energy from their next turn. A player discovers that borrowing just before a reset avoids repayment. RuleForge finds the sequence, shows the energy balance after each move, and proposes ways to close the loophole. It then checks whether the fix accidentally makes normal play impossible.

What the user does

1. **Describe.** The designer enters a rule and confirms its precise interpretation.
2. **Play and explore.** A playable build appears. Agents explore strategies under a fixed compute budget.
3. **Inspect.** An exploit appears as a short replay with the violated rule highlighted.
4. **Revise.** The designer approves a correction and reruns the original exploit plus new tests.

Who would use it and why

Indie studios and tabletop or digital-game designers who repeatedly prototype mechanics. Their current alternative is manual playtesting, scripts, or existing game-testing software.

How it could become a business

Test a paid workspace with metered simulation runs. Start by selling a small design-team pilot. Payment depends on finding useful issues or shortening iteration time; an entertaining demo alone does not establish demand.

Existing work and proposed difference

The proposed distinction is the complete cycle from rule intent to executable behavior, exploit discovery, replay, and designer-controlled revision. modl.ai already offers AI game testing [S02]. Do not claim that agent playtesting itself is new.

CANDIDATE 1 | BUILD AND EVIDENCE

RuleForge in practice

SUGGESTED TRACK: Coding and Agentic Engineering

Rule description -> reviewed rule specification -> constrained game engine -> parallel agent matches -> exploit replay -> approved revision -> regression run

What the AI does; what the software checks

Nemotron interprets rule intent, proposes adversarial strategies, and explains concrete failures. Deterministic game code tracks resources and validates moves. Nebius workers run batches of matches. Begin with API actions rather than costly screenshot-based gameplay.

Smallest complete hackathon version

One polished turn-based resource game; a limited rule language; three exploit families such as resource loops, unreachable objectives, and dominant strategies; versioned replays; a working editor and play button.

Keep outside the first version

No arbitrary video-game generation, universal game engine, or claim that bots can measure human enjoyment. A rule is not automatically an exploit simply because an agent dislikes it.

A three-minute demo plan

Time	What to show
0:00-0:25	Show a designer entering a previously unseen rule.
0:25-1:15	Generate the supported implementation and run agents; show progress honestly.
1:15-2:10	Replay the exploit and inspect the exact resource or state violation.
2:10-3:00	Approve a repair, rerun checks, and show a human playing the result.

How to prove it works

Measure useful exploit discovery against random play and a scripted baseline, time to first confirmed finding, false alarms, and performance on held-out rule changes. Ask designers to assess whether the findings would change their design.

Main failure risk

A scripted demonstration can look impressive without discovering anything. Keep unseen rule changes and saved execution traces. If exploration takes longer than the video, label recorded or accelerated segments.

The first 48-hour experiment

Within 48 hours, implement one rule family and see whether the system discovers a non-hardcoded exploit on an unseen variation. Stop or narrow if it cannot outperform a simple baseline.

CANDIDATE 2 | PRODUCT AND OPPORTUNITY

Blackout

Team coordination that survives loss of internet access

Disconnect the internet. Disconnect the coordinator. The team can still work.

In plain language

Blackout turns voice notes, photographs, and messages into a shared operations board for a team with unreliable connectivity. Nearby devices exchange updates through a supported local network. The system explains what it knows, which information is old, and where people disagree.

A concrete example

At a film shoot, the generator delivery is delayed. One group records a new arrival time while disconnected; another group reallocates the backup generator. When they reconnect, Blackout links both updates, identifies the conflicting allocation, and asks the responsible person to resolve it. It does not quietly invent a single agreed plan.

What the user does

1. **Prepare.** Choose a site, participants, roles, and the information available offline.
2. **Capture.** People send observations; the system proposes structured tasks and dependencies.
3. **Continue.** During disconnection, users consult cached evidence and exchange reachable local updates.
4. **Reconcile.** On reconnection, new facts merge and disputed commitments are surfaced.

Who would use it and why

Film crews, event operators, field-service teams, and remote sites with frequent connectivity problems. Begin with routine operations, not emergency dispatch or medical decisions.

How it could become a business

Test a paid deployment for one event or site, then recurring team/site pricing. Buyers must value reduced coordination errors enough to justify installation, training, and support.

Existing work and proposed difference

Briar already supports offline messaging over Bluetooth or Wi-Fi [S03]. The proposed distinction is structured, evidence-linked operational coordination, freshness indicators, and explicit reconciliation. This is not a claim that offline communication is novel.

CANDIDATE 2 | BUILD AND EVIDENCE

Blackout in practice

SUGGESTED TRACK: Best Apps and Agents

Observed update -> evidence record -> proposed task/fact -> local event log -> reachable peer sync -> conflict review -> shared board

What the AI does; what the software checks

Use Nemotron on Nebius for connected multimodal intake, task interpretation, and preparation of compact site knowledge. A supported local open model on a laptop handles bounded offline queries. Phones can initially be capture clients; do not assume every phone can run the model.

Smallest complete hackathon version

Three or four devices on a local network; one laptop with local inference; one operations scenario; structured tasks, evidence links, stale-state warnings, disconnection handling, and reconciliation after reconnecting.

Keep outside the first version

No claim that disconnected devices instantly know each other's state. No universal Bluetooth mesh, autonomous critical dispatch, or need to split an LLM across phones. Use a proven transport and a narrow synchronization protocol.

A three-minute demo plan

Time	What to show
0:00-0:30	Show the shared board and a voice/photo update becoming a proposed task.
0:30-1:15	Disconnect WAN internet while local communication continues.
1:15-2:10	Partition a participant and create conflicting updates; show local-only knowledge.
2:10-3:00	Reconnect, resolve the conflict, and show the synchronized evidence history.

How to prove it works

Measure update delivery after reconnection, data loss, duplicate actions, conflict detection, task correctness, and answer freshness. Compare with a shared chat plus checklist on the same scenario.

Main failure risk

No protocol can provide immediate global agreement across unreachable groups. Require approval for disputed allocations. Coordinator failure may preserve the board while AI capability degrades; show that honestly rather than claiming uninterrupted intelligence.

The first 48-hour experiment

Within 48 hours, disconnect and reconnect three devices without losing a structured update, and correctly surface one conflicting assignment. Verify local-model memory and latency before investing in the UI.

Chimera

Ordinary devices assemble into a temporary inspection system

Turn a few phones and a laptop into one coordinated AI instrument.

In plain language

Chimera gives enrolled devices different jobs: one phone watches the front of an object, another watches the side, and a laptop processes observations. It reasons about what can actually be seen and asks for a better viewpoint when an important detail is hidden.

A concrete example

A training workshop assembles a small kit. One camera can see the parts tray; another can see the connector. Chimera checks supported steps. If the connector camera disconnects, it stops claiming that the connection is verified and guides someone to reposition the remaining camera.

What the user does

1. **Enroll.** Users explicitly join devices and grant camera or microphone access.
2. **Calibrate.** The system identifies camera positions and a small set of inspection targets.
3. **Inspect.** Observations become task evidence; uncertain steps remain unverified.
4. **Recover.** A device disappears; the system proposes a new arrangement and recalibrates.

Who would use it and why

Small assembly teams, vocational workshops, and organizations needing temporary inspection stations. A specific inspection task is essential; a generic device platform is harder to sell.

How it could become a business

Test a paid inspection workflow or workshop deployment. Recurring revenue could come from maintained task templates, device administration, and repeated inspection sessions.

Existing work and proposed difference

Device pooling is not new. exo explores distributed inference across everyday devices [S04]. Chimera's proposed contribution is coordinating observation, coverage, calibration, and evidence for one useful task, rather than pooling GPU memory.

CANDIDATE 3 | BUILD AND EVIDENCE

Chimera in practice

SUGGESTED TRACK: Physical AI or Best Apps and Agents, based on the implemented system

Device capabilities -> user-approved roles -> calibration -> synchronized observations -> coverage checks -> inspection evidence -> recovery plan

What the AI does; what the software checks

An NVIDIA vision-capable open model, selected after a smoke test, interprets supported observations; Nemotron plans device roles and explains uncertainty. Nebius handles heavier processing. Local capture buffers short interruptions. Calibration and timestamp checks use conventional algorithms.

Smallest complete hackathon version

Two phone cameras and one laptop; a marker-assisted tabletop setup; one simple kit assembly; a small set of observable steps; an interface showing camera coverage; recovery from one camera loss.

Keep outside the first version

No arbitrary-device interoperability, fully automatic calibration in every room, or replacement of certified industrial inspection. Do not infer an invisible connection from the fact that someone moved their hand near it.

A three-minute demo plan

Time	What to show
0:00-0:30	Enroll two phones and assign viewpoints.
0:30-1:15	Perform the supported assembly and show evidence-linked checks.
1:15-2:00	Remove one camera; the system marks the lost observation as unverified.
2:00-3:00	Follow a repositioning instruction, recalibrate, and resume verification.

How to prove it works

Measure missed steps, false confirmations, calibration time, recovery time, and performance under occlusion or lighting changes. Compare with a single-camera setup; the extra devices must improve something.

Main failure risk

Poor calibration and hidden details can make the entire workflow unreliable. Keep the first task visually observable and tolerate explicit uncertainty. Obtain consent for capture and limit retention.

The first 48-hour experiment

Within 48 hours, demonstrate that a second view resolves one ambiguity the first view cannot, then recover after losing that view. If the multi-device benefit is marginal, narrow the task or stop.

CANDIDATE 4 | PRODUCT AND OPPORTUNITY

IncentiveLab

Adversarial agents test whether business rules can be exploited

Before launching a reward program, let agents try to game its rules.

In plain language

A team describes a referral, loyalty, or booking policy. IncentiveLab creates a sandbox with those rules and lets agents search for profitable sequences of allowed actions. Every claimed exploit must execute successfully in that sandbox and produce an inspectable trace.

A concrete example

A referral scheme awards credits when a purchase is made but does not reverse them after a refund. Agents discover that a sequence of purchases, referrals, and refunds generates unearned credits. A repair closes the loop, but a second test reveals it also blocks a legitimate family account scenario.

What the user does

1. **Specify.** Confirm the exact policy, goals, and intended legitimate behaviors.
2. **Attack in a sandbox.** Agents try strategies using synthetic accounts and transactions.
3. **Replay.** Inspect the action sequence, balances, and violated business condition.
4. **Compare repairs.** Check candidate fixes against exploit cases and legitimate-user cases.

Who would use it and why

Marketplaces and subscription businesses with nontrivial promotions or allocation policies. The first user is likely a product, fraud, or operations team working with engineering.

How it could become a business

Start with a scoped policy review using synthetic data. A recurring offering could rerun checks whenever promotions or policy code change. Pricing must reflect avoided review work and useful findings, not speculative fraud-loss claims.

Existing work and proposed difference

Microsoft's Magentic Marketplace already studies agents in simulated markets [S05]. The proposed product difference is translating a bounded real policy into an executable sandbox, producing concrete abuse traces, and evaluating repairs.

CANDIDATE 4 | BUILD AND EVIDENCE

IncentiveLab in practice

SUGGESTED TRACK: Best Apps and Agents

Reviewed policy -> executable state machine -> adversarial strategies -> sandbox transactions -> confirmed exploit -> repair comparison

What the AI does; what the software checks

Nemotron proposes diverse strategies and maps policy language to a draft specification. Engineers approve the interpretation. Deterministic code executes all transactions and checks balances or entitlements. Nebius workers explore strategies in parallel.

Smallest complete hackathon version

One referral or credits program; a few supported operations; synthetic identities; two or three abuse patterns; an exploit replay; and a comparison showing effects on legitimate cases.

Keep outside the first version

No attacks on live businesses, real payments, forecasts of human adoption, or claims to predict total fraud losses. A simulated strategy is an executable counterexample under stated rules, not a prediction of customer behavior.

A three-minute demo plan

Time	What to show
0:00-0:25	Show a simple promotion that appears reasonable.
0:25-1:15	Run varied agents and surface a confirmed balance or entitlement violation.
1:15-2:05	Replay the exact sequence with synthetic transaction records.
2:05-3:00	Compare two repairs, including their effect on a legitimate user scenario.

How to prove it works

Measure confirmed exploits, false reports, diversity of attack sequences, compute per finding, and legitimate scenarios broken by a repair. Compare against random actions and a fixed checklist.

Main failure risk

An inaccurate sandbox can manufacture fictitious exploits. The policy owner must review assumptions; report unsupported rules. Generalized fraud claims would overstate the evidence.

The first 48-hour experiment

Within 48 hours, recover an independently defined loophole from a reviewed policy without encoding its answer in the agent prompt. Show that a repair prevents the replay.

Morph

A simulated robot recovers after a supported change to its body

Change the robot. Watch it search for a new way to finish the task.

In plain language

Morph starts with a simulated robot that can perform a simple task. A user disables an actuator, changes a limb parameter, or adds a payload. The system detects the supported change, evaluates alternative control strategies, and tries to recover useful behavior.

A concrete example

A small simulated vehicle normally reaches a target with all actuators available. One actuator is disabled. The old controller fails. Morph searches a bounded controller space and finds a slower but successful route, then evaluates it on target positions not used during recovery.

What the user does

1. **Establish a baseline.** Show the intact robot completing the task.
2. **Change the body.** Apply a supported fault or parameter change.
3. **Search.** Run candidate recovery strategies in parallel and expose failed attempts.
4. **Validate.** Test the selected controller on held-out starts, targets, and disturbances.

Who would use it and why

Robotics developers studying configurable platforms or degraded operation. Real commercial use would require convincing performance on their own hardware and workloads.

How it could become a business

Begin with a paid evaluation or integration engagement. A recurring platform could support regression testing across robot configurations, but that market claim needs validation and real deployment evidence.

Existing work and proposed difference

Morphology-conditioned control is established research, including MetaMorph [S06]. The proposed distinction is an interactive recovery workflow with explicit supported changes, measurable recovery cost, and independently replayable results.

CANDIDATE 5 | BUILD AND EVIDENCE

Morph in practice

SUGGESTED TRACK: Physical AI

Task and body state -> supported change detection -> candidate recovery plans -> parallel simulation -> controller selection -> held-out evaluation

What the AI does; what the software checks

Use an NVIDIA model to interpret goals, choose recovery experiment families, and explain evidence. A numerical controller handles fast actions. Nebius GPUs run simulation or optimization. Verify simulator, model, and infrastructure compatibility before committing.

Smallest complete hackathon version

One simple simulated robot; one task; two supported changes; a bounded controller family; replayable trials; recovery-time and success-rate displays. Simulation alone can be a complete, honestly labeled prototype.

Keep outside the first version

No universal zero-shot recovery, arbitrary damaged humanoids, or claim that simulated success proves physical safety. Do not spend the deadline training a general robot foundation model.

A three-minute demo plan

Time	What to show
0:00-0:25	Show intact behavior and the baseline score.
0:25-0:50	Disable a supported actuator; show the original controller failing.
0:50-2:00	Show a transparent recovery search and selected controller.
2:00-3:00	Run held-out trials and report success, recovery cost, and remaining failures.

How to prove it works

Compare with a fixed controller and a simple search baseline. Measure recovery time, task success across seeds, compute cost, and performance on held-out changes within the supported family.

Main failure risk

Recovery may be too slow or too unreliable for a live demonstration. Label prerecorded search or time compression. Generalization is a research challenge; a single lucky trial is not sufficient.

The first 48-hour experiment

Within 48 hours, recover one simple behavior after a supported change using a conventional optimizer. Only then add model-guided experiment selection and compare its contribution.

CANDIDATE 6 | PRODUCT AND OPPORTUNITY

Treaty

Agents negotiate commitments on behalf of separate people

Give each person an agent. Reach an agreement without surrendering control.

In plain language

Each participant enters their constraints and decides which information their agent may disclose. Agents exchange proposals for a shared plan. A separate checker verifies the proposed commitments, and affected people approve the final agreement.

A concrete example

Five teams share a projector, a vehicle, and several time slots. One team can trade its projector time for transport; another has a hard deadline. Treaty proposes an exchange. When a vehicle becomes unavailable, it identifies which commitments must change and renegotiates instead of pretending the old plan still works.

What the user does

1. Delegate. Each participant confirms priorities, hard constraints, and disclosure rules.

2. Propose. Agents exchange permitted proposals within a bounded negotiation protocol.

3. Verify. A solver checks that the proposed plan satisfies the encoded constraints.

4. Commit. People approve commitments; later changes require appropriate reapproval.

Who would use it and why

Shared laboratories, production teams, and equipment networks coordinating independent parties. The first use case must contain meaningful negotiation beyond ordinary calendar scheduling.

How it could become a business

Test a paid coordination workflow for one organization or resource network. Possible recurring value comes from repeated allocations, approval records, and integrations, not the novelty of agents chatting.

Existing work and proposed difference

Multi-agent negotiation and user-interest evaluation already exist in Microsoft research [S07, S08]. The proposed product contribution is bounded delegation, explicit commitments, understandable trade-offs, and controlled renegotiation.

CANDIDATE 6 | BUILD AND EVIDENCE

Treaty in practice

SUGGESTED TRACK: Best Apps and Agents

Individual constraints -> approved disclosure -> bounded proposals -> feasibility checks -> individual approvals -> commitment record -> renegotiation

What the AI does; what the software checks

Nemotron interprets preferences and proposes trade-offs. A conventional solver checks encoded hard constraints. Nebius runs participant agents with separated contexts and access controls. These controls do not themselves prove privacy against inference from proposals.

Smallest complete hackathon version

Three to five participants; a fixed resource-allocation problem; hard and soft constraints; a small proposal vocabulary; explicit approval; infeasibility explanations; one resource-withdrawal scenario.

Keep outside the first version

No legal contract generation, autonomous purchases, guaranteed negotiation fairness, or cryptographic privacy claims without a proven protocol. Separate draft proposals from binding application commitments.

A three-minute demo plan

Time	What to show
0:00-0:35	Give participants different constraints and show their approved delegation limits.
0:35-1:20	Agents propose an allocation; expose a useful trade-off.
1:20-2:05	Verify the plan and collect approvals.
2:05-3:00	Withdraw one resource; renegotiate or correctly explain why no feasible plan remains.

How to prove it works

Measure hard-constraint violations, unauthorized disclosures or commitments, negotiation rounds, and objective value against a centralized solver on synthetic cases. Assess whether users understand what they approved.

Main failure risk

The agents may add cost to a problem a solver already handles. Their value must be interpreting messy preferences or communicating useful trade-offs. Refusing an impossible agreement is a correct result.

The first 48-hour experiment

Within 48 hours, solve a small allocation with plain language constraints, prevent an unauthorized concession, and explain one infeasible case. Compare setup effort with a simple form plus solver.

CANDIDATE 7 | PRODUCT AND OPPORTUNITY

EventWeave

Evidence-linked timelines that preserve unresolved uncertainty

Combine conflicting recordings. Show what happened, what might have happened, and what is still missing.

In plain language

EventWeave combines recordings and written accounts into an inspectable timeline. It aligns observable landmarks, proposes clock offsets, and links every extracted event to its source. When several explanations fit, it keeps them separate instead of generating a confident story.

A concrete example

Three phones record different parts of an equipment handover. One clock is several minutes wrong, and a note claims an inspection happened before delivery. A visible shared event helps align the recordings. The system shows which ordering the evidence supports and which part remains unobserved.

What the user does

1. Import. Upload consented recordings and accounts; preserve originals and identifiers.

2. Align. Review detected landmarks, timestamp offsets, and proposed event matches.

3. Compare. Inspect consistent timelines and contradictions with clickable source segments.

4. Resolve. Add a requested missing observation or leave the question explicitly unresolved.

Who would use it and why

Industrial operations teams reviewing handovers or equipment incidents. Start with a narrow internal-review workflow rather than criminal adjudication or automated blame assignment.

How it could become a business

Test a paid review workspace or scoped incident-review pilot. Recurring value depends on reducing review effort while preserving evidence quality; incumbents make differentiation demanding.

Existing work and proposed difference

Axon already provides substantial video investigation and timeline tooling [S09]. The proposed distinction is preserving alternative explanations and requesting specific missing evidence. A searchable video timeline alone is not enough.

CANDIDATE 7 | BUILD AND EVIDENCE

EventWeave in practice

SUGGESTED TRACK: Best Apps and Agents

Original evidence -> extracted events -> reviewed alignment -> temporal constraints -> consistent alternatives -> source-linked explanation -> missing-evidence request

What the AI does; what the software checks

NVIDIA open models extract candidate events and interpret accounts on Nebius. Deterministic temporal checks validate ordering constraints. A timestamp offset remains a hypothesis until supported; source authenticity is not established merely by ingesting a file.

Smallest complete hackathon version

A consented staged handover; three short recordings; controlled clock offsets; one misleading account; source-linked event cards; alternative orderings; and an explicit unresolved state.

Keep outside the first version

No definitive causal conclusions from timing alone, facial identification, truth-detection claims, automatic liability assignment, or forensic-certification claims. Preserve originals and distinguish observations from interpretations.

A three-minute demo plan

Time	What to show
0:00-0:30	Show conflicting accounts of a staged handover.
0:30-1:20	Align a shared observable landmark across recordings.
1:20-2:10	Expose the unsupported ordering and show surviving explanations.
2:10-3:00	Add one withheld clip; update the timeline and retain a source trail.

How to prove it works

Use staged ground truth to measure alignment error, event extraction precision, unsupported claims, and correct abstention. Ask another person to reproduce each conclusion from the linked evidence.

Main failure risk

An incorrect match can contaminate the whole timeline. Require review for uncertain alignments and use explicit uncertainty. Strong incumbents and trust requirements make this the hardest first-market entry on the shortlist.

The first 48-hour experiment

Within 48 hours, align three short clips with known offsets and correctly leave an intentionally unobserved event unresolved. Stop if the system confidently invents missing events.

TEAM DECISION WORKSHEET

Choose with evidence

Have each teammate read the two pages for their preferred idea. Shortlist two, run one small technical experiment for each, and then commit. Avoid building two full projects in parallel.

Question	Candidate A	Candidate B
Name and owner	_____	_____
Core experiment completed?	_____	_____
What result surprised us?	_____	_____
Which real user can test it?	_____	_____
What can we show in 3 minutes?	_____	_____
What existing tool must we beat?	_____	_____
Biggest unresolved blocker	_____	_____
Team preference and reason	_____	_____

Suggested schedule from 14 October

14-16 October: test the core mechanism, speak with potential users, and choose.
17-21 October: build one end-to-end user journey and record failures.
22-25 October: test unfamiliar cases, compare baselines, and improve the product with user feedback.
26-28 October: prepare the video, public repository, setup instructions, and hosted judge experience.
29-30 October: verify access from a fresh device and submit with buffer. The deadline is 30 October at 10:30 PM IST [S01].

Before calling an idea the winner

Require a working core mechanism, a believable user, a meaningful NVIDIA/Nebius contribution, and a measurable result beyond a simple baseline. If the basic mechanism fails, narrow or change it early. Do not hide a failure behind prerecorded success or invented benchmarks.

Current recommendation

Choose **RuleForge** if you want the best balance of a memorable, interactive demo and a finishable software product. Choose **Blackout** if the team prefers distributed systems and can access an operational pilot. Keep **Chimera** as the strongest alternative for a physical demonstration using basic devices.

SOURCES AND RESEARCH LIMITS

References 1 of 2

Source claims were checked during the research for this conversation. This guide is dated 14 October 2026. Product capabilities and event details can change; verify before implementation or submission.

[S01] Nebius x NVIDIA Global AI Hackathon - official overview

Prize, deadline, tracks, submission requirements, and published judging criteria. Follow the Rules link for the full terms.

Open source

[S02] modl.ai - company and AI game testing

Evidence that agent-based game testing is an existing category. It does not validate RuleForge's proposed differentiation.

Open source

[S03] Briar - how it works

Offline messaging over supported Bluetooth and Wi-Fi connections; establishes a transport-layer precedent.

Open source

[S04] exo - archived distributed-inference repository

Prior work pooling everyday devices for inference. Archived repository; consult the active project before any implementation reuse.

Open source

[S05] Microsoft Research - Magentic Marketplace

An open-source environment for studying interactions between agents in simulated markets.

Open source

[S06] MetaMorph - Learning Universal Controllers with Transformers

Prior research on controllers conditioned on robot morphology. Does not establish Morph's proposed recovery performance.

Open source

[S07] Microsoft Research - SocialReasoning-Bench

Prior work evaluating agent behavior in scheduling and marketplace negotiation.

Open source

SOURCES AND RESEARCH LIMITS

References 2 of 2

Source claims were checked during the research for this conversation. This guide is dated 14 October 2026. Product capabilities and event details can change; verify before implementation or submission.

[S08] Microsoft Research - The Treacherous Envoy Problem

Research discussing tensions between expressive negotiation, verifiable intent, and bounded disclosure.

Open source

[S09] Axon Investigate

Existing video investigation tools and workflows; an important adjacent incumbent for EventWeave.

Open source

[S10] Nancy Agarwal - NVIDIA author profile

Public technical background used to inform the emphasis on practical integration; not evidence of voting preferences.

Open source

[S11] FACTS About Building Retrieval Augmented Generation-based Chatbots

Research coauthored by Abhinav Balasubramanian, among others; supports attention to grounded, evaluated systems.

Open source

[S12] Nebius Data Lab - Your best dataset is already in your logs

Public work informing the emphasis on learning from operational traces and measuring improvements.

Open source

[S13] NVIDIA-Verified Agent Skills Provide Capability Governance for AI Agents

Public work informing the emphasis on constrained capabilities, evaluation, and traceable actions.

Open source

How to interpret the research

This is a decision guide, not an exhaustive market study. Competitor references establish overlap; they do not establish the absence of other competitors. Business models and buyer descriptions are hypotheses. No sales, benchmark results, prototype capabilities, judge endorsements, or winning probabilities are claimed.

Model and infrastructure selection

The plans specify responsibilities rather than unverified exact model configurations. Before selecting an idea, confirm the chosen NVIDIA open model's capabilities, license, serving availability, hardware requirements, and Nebius costs with a small working test.